

FAST: Full-stack Agentic Co-design for Dynamic Sparse Attention

Genhao Zhang, Zishen Wan
Columbia University, New York, USA
{gz2406, zw3306}@columbia.edu

I. OVERVIEW OF THE TASK ADDRESSED

While dynamic sparse attention mitigates the $O(N^2)$ complexity of self-attention, its irregular, data-dependent sparsity causes unpredictable memory access and processing-element (PE) underutilization. [1] Existing domain-specific architectures (DSAs) [1, 2, 3] rely on static, algorithm-specific co-design, limiting adaptability. To address this, we propose FAST, an agentic full-stack loop. Rather than merely orchestrating CHIA [4] tools, FAST leverages structured cross-layer feedback to attribute bottlenecks directly to the algorithm, compiler, or microarchitecture, enabling targeted adaptation.

II. METHODOLOGY: LOOP DESIGN AND EXECUTION PLAN

A. The Agentic Co-Design Loop

Built on the CHIA framework [4], as illustrated in **Fig. 1**, FAST orchestrates specialized agents in a closed-loop co-design architecture. To guarantee hardware validity, FAST decouples search driven by agents from generation. The Compiler and μ Arch agents jointly explore the coupled software-hardware space under continuous diagnostic feedback across three phases:

- **Workload Characterization (Kernel Agent):** Configures dynamic attention hyperparameters to extract sparse-index profiles and map the sparsity-accuracy Pareto frontier within target design constraints.
- **Hardware-Software Co-Optimization (Compiler & μ Arch Agents):** Jointly optimizes software scheduling parameters and hardware parameters. The μ Arch Agent applies Retrieval-Augmented Generation (RAG) over verified Chisel templates to synthesize synthesizable RTL, validated via Verilator simulations.
- **Evaluation & Diagnostic Feedback (Evaluator):** Performs multi-fidelity evaluation via Verilator for fast functional checks, FireSim for high-fidelity performance, and Hammer for power, performance, and area (PPA). The Critic Agent attributes bottlenecks to the algorithm, compiler, or microarchitecture, dispatching layer-specific mutation vectors for the next iteration.

B. Execution Plan

The project spans four phases over the hackathon timeline:

- (1) **Baseline:** Integrate Compiler and μ Arch agents into CHIA.(Sep. 4)
- (2) **Pipeline:** Implement Kernel Agent, Chisel RAG, and Critic for structured cross-layer attribution.(Sep. 11)

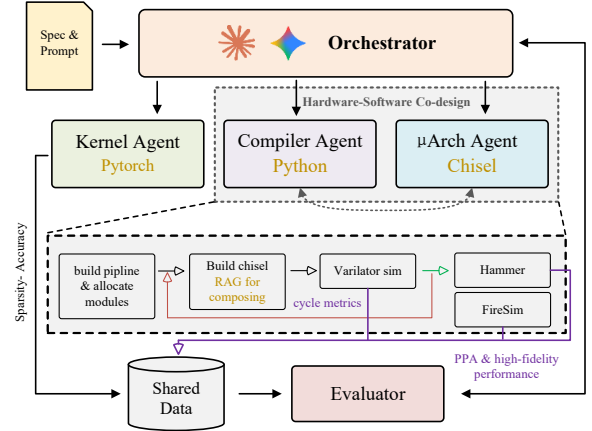


Fig. 1. Overview of the FAST framework. Agents operate via structured I/O loops; detailed specifications (inputs, actions, outputs, feedback, and stopping criteria) are provided in **Appendix**

- (3) **Design Space Exploration (DSE):** Execute Verilator DSE with black-box baselines and no-Critic ablations.(Sep. 18)
- (4) **Validation:** Conduct FireSim simulation and Hammer PPA extraction; open-source framework.(Sep. 20)

III. EXPECTED RESULTS

We expect to deliver the following outcomes:

- An open-source, composable agentic loop integrated into CHIA for reproducible full-stack co-design.
- The sparsity-accuracy-PPA Pareto front mapped via closed-loop search, rigorously validated against hand-tuned experts, equal-budget black-box DSE, and a no-Critic ablation on independent golden tests.
- Quantitative success criteria: demonstrating measurable gains in PE utilization and EDP over black-box DSE, with negligible accuracy loss against dense models.

IV. COST ESTIMATE

Table I details the estimated budge as following:

TABLE I
ESTIMATED CLOUD AND API BUDGET.

Resource	Usage	Rate	Est. cost
GCP CPU (Verilator DSE)	~300 h	\$0.55/h	\$165
GCP GPU (PyTorch eval)	~150 h	\$0.75/h	\$112
GCP FPGA (FireSim)	~30 h	\$6.00/h	\$180
Gemini API (Evaluator)	~70M tokens	blended	\$700
Storage & misc.	—	—	\$50
Total			~\$1,207

TABLE II
DETAILED MODULE SPECIFICATIONS FOR THE FAST FRAMEWORK.

Module	Inputs	Actions	Outputs	Feedback	Stopping Criteria
Orchestrator	Specs, constraints	Build pipeline	Prompts, shared state	Completion signals	Pipeline initialized
Kernel	Weights, acc. bound	Tune sparsity params	Sparse-index profiles	PyTorch accuracy	Acc. drop $< \epsilon$
Compiler	Sparse profiles, specs	Mem./dataflow sched.	Compiler IR, schedule	Verilator cycles	Reduction or timeout
μ Arch	Schedule, arch specs	Compose Chisel modules	Synthesizable RTL	Verilator simulation	Functional pass
Critic & Eval.	RTL, workload, metrics	Attribute bottlenecks	Layer-specific mutations	PPA, cycles, accuracy	Resolved or max iter.

REFERENCES

- [1] X. Xiong, Z. Chen, Y. Liang, M. Tian, J. Shang, J. Zhong, and D. Liu, “Dynax: Sparse attention acceleration with dynamic x: M fine-grained structured pruning,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 260–274.
- [2] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, “Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
- [3] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE international symposium on high-performance computer architecture (HPCA)*. IEEE, 2021, pp. 97–110.
- [4] A. Cui, F. Hermida-Rivera, J. Toubes, R. Gupta, J. Fang, C. L. Zhang, E. Schwarz, J. Kim, Y. S. Shao, B. Nikolic *et al.*, “Chia: An open-source framework for principled, agentic ai-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.

APPENDIX

Agent Specifications. Table II details the inputs, actions, outputs, feedback sources, and stopping criteria for each agent. The μ Arch Agent guarantees hardware validity by *composing verified Chisel modules* rather than unrestricted RTL generation.